

# Quantum Research Implementation and Verification Plan

## Evidence plan for the 0.3.0 research design

Phexora AI · phexora.ai

2026-07-21

### Abstract

This document turns the Quantum research requirements into a human-owned, dependency-ordered implementation and verification plan. It defines reviewer roles, evidence classes, task gates, adversarial scenarios, performance reporting, and release criteria for a candidate private post-quantum DAG protocol. It is a planning and assurance document; it does not report a conformant implementation or completed verification.

**Keywords:** post-quantum cryptography; protocol verification; evidence gates; reproducible benchmarking; independent review.

## 1. Purpose and status

This guide turns the Quantum research requirements into an ordered body of work. It does **not** certify the design, prove security, or report completed tests. At this revision:

- there is no protocol implementation in this repository;
- none of the task acceptance gates has passed;
- no benchmark has been run;
- no external cryptographic or consensus review has been completed;
- there is no conformant testnet or production network.

The goal is a protocol that simultaneously satisfies:

1. end-to-end post-quantum security;
2. private ledger semantics and network anonymity by default; and
3. at least 1,000 accepted layer-1 transactions per second.

These are hard release requirements. They are not current capability claims. The initial product boundary is private post-quantum cash and settlement with bounded payment policies, not a general-purpose smart-contract platform.

## 2. Governance and reviewer roles

AI systems may help draft code, tests, vectors, and analyses. They **MUST NOT** approve their own cryptographic design, formal argument, benchmark, audit finding, or release.

Every task has:

- an accountable human owner;
- an independent reviewer who did not author the work;
- explicit dependencies;
- versioned artifacts;
- a pass/fail decision with evidence;
- unresolved-risk and change-control records.

The minimum specialist review roles are:

| Role                  | Approval boundary                         |
|-----------------------|-------------------------------------------|
| Cryptography reviewer | primitives, parameters, KATs, composition |
| Proof-system reviewer | AIR, transcript, FRI, ZK, soundness       |

| Role                             | Approval boundary                                            |
|----------------------------------|--------------------------------------------------------------|
| Consensus reviewer               | GHOSTDAG profile, DAA, ordering, finality                    |
| Network-privacy reviewer         | observer model, routing, traffic analysis                    |
| Implementation-security reviewer | parsers, side channels, supply chain                         |
| Performance reviewer             | workload, topology, measurement validity                     |
| Legal counsel                    | launch, token, privacy, sanctions and jurisdiction questions |

No single reviewer may sign all technical boundaries.

### 3. Evidence taxonomy

Reports MUST label evidence accurately:

| Evidence type          | What it can support                                  | What it cannot support alone                       |
|------------------------|------------------------------------------------------|----------------------------------------------------|
| Analytical proof       | a stated theorem under stated assumptions            | implementation correctness                         |
| Formal verification    | properties encoded and proved in a named formal tool | omitted properties or real hardware behavior       |
| KAT/conformance vector | exact algorithm and encoding agreement               | general security                                   |
| Unit/property test     | exercised implementation behavior                    | negligible failure probability or a reduction      |
| Differential/fuzz test | cross-implementation mismatch and parser defects     | absence of defects                                 |
| Statistical experiment | empirical distribution or attack result              | binding, zero knowledge, or anonymity as a theorem |
| Benchmark              | performance for a pinned environment/workload        | performance on other environments                  |
| Code review            | implementation findings                              | cryptographic proof                                |
| External audit         | independent review within its stated scope           | guarantees outside that scope                      |

Miri, sanitizers, linters, tests, and manual review are valuable assurance tools but MUST NOT be called formal verification. One million collision-free trials do not prove commitment binding. A prover test with no observed false acceptance does not establish  $2^{128}$  soundness. Uniform-looking ciphertext bytes do not prove privacy.

### 4. Requirements traceability

| Requirement              | Primary tasks                                     | Required release evidence                                 |
|--------------------------|---------------------------------------------------|-----------------------------------------------------------|
| R1 Private ledger        | T103–T104, T301–T305, T401–T405                   | security argument, vectors, two implementations, audit    |
| R2 Network anonymity     | T601–T602, T604                                   | threat model, simulation, adversarial experiments, review |
| R3 Post-quantum security | T001, T103–T104, T201–T204, T303–T305, T601, T701 | composed 128-bit PQ analysis and KATs                     |
| R4 Authorization/supply  | T202, T301, T305, T402–T405, T504                 | no-inflation proof/tests and issuance-cap proof           |
| R5 Consensus safety      | T403, T501–T504, T603                             | deterministic models, vectors, reorg/recovery tests       |

| Requirement             | Primary tasks                             | Required release evidence                                         |
|-------------------------|-------------------------------------------|-------------------------------------------------------------------|
| R6 Scalability          | T004, T304–T305, T603–T604                | pre-node feasibility and 1,000 accepted tx/s end-to-end artifacts |
| R7 Setup/upgrade safety | T002, T303–T304, T501, T701               | setup analysis, version/replay/upgrade review                     |
| R8 Verifiability        | T003–T004, all implementation tasks, T701 | trace report, independent implementations, external review        |

A task is not complete if its output is not linked from this matrix through a machine-readable evidence manifest.

## 5. Acyclic task graph

Tasks MUST be completed in dependency order. A later experiment may send an earlier task back to design, but dependencies MUST NOT be bypassed.

### Layer 0 - Scope and evidence

|                                                  |               |
|--------------------------------------------------|---------------|
| T001 Security profile and threat model           |               |
| T002 Canonical serialization and domain registry | <- T001       |
| T003 Test-vector and interoperability harness    | <- T002       |
| T004 Benchmark and artifact harness              | <- T001, T002 |

### Layer 1 - Algebra and commitment

|                                             |                                       |
|---------------------------------------------|---------------------------------------|
| T101 Field and extension-field arithmetic   | <- T001, T003                         |
| T102 Polynomial/NTT layer, only if selected | <- T101                               |
| T103 Samplers and commitment candidate      | <- T001, T002, T101; T102 if required |
| T104 Commitment security gate               | <- T003, T004, T103                   |

### Layer 2 - Standardized cryptography and keys

|                                               |               |
|-----------------------------------------------|---------------|
| T201 SHAKE256 and domain hashing              | <- T002, T003 |
| T202 SLH-DSA authorization                    | <- T201       |
| T203 ML-KEM, KDF and authenticated encryption | <- T201       |
| T204 Wallet key hierarchy and address profile | <- T202, T203 |

### Layer 3 - Validity proof

|                                                  |                           |
|--------------------------------------------------|---------------------------|
| T301 Integer transaction relation and AIR        | <- T101, T104, T201, T202 |
| T302 Membership, nullifier and state AIR         | <- T301                   |
| T303 Fiat-Shamir, FRI and zero-knowledge profile | <- T101, T201             |
| T304 Prover, verifier and aggregation            | <- T301, T302, T303       |

### Layer 4 - Transactions and state

|                                                        |                                 |
|--------------------------------------------------------|---------------------------------|
| T401 Note creation and recipient encryption            | <- T104, T203, T204             |
| T305 Cryptographic transaction feasibility gate        | <- T004, T202, T204, T304, T401 |
| T402 Transaction codec, builder and authorization hash | <- T202, T301, T305, T401       |
| T403 Commitment/nullifier state and atomic application | <- T302, T402                   |
| T404 Wallet scanning and spending                      | <- T204, T401, T402, T403       |
| T405 Selective disclosure and bounded payment policies | <- T204, T304, T404             |

### Layer 5 - Consensus and issuance

|                                           |                     |
|-------------------------------------------|---------------------|
| T501 Versioned GHOSTDAG consensus profile | <- T001, T002       |
| T502 Header, PoW and deterministic DAA    | <- T201, T501       |
| T503 Canonical DAG-to-state ordering      | <- T403, T501, T502 |
| T504 Rewards, supply cap and genesis      | <- T503             |

### Layer 6 - Network and integration

|                                                  |                           |
|--------------------------------------------------|---------------------------|
| T601 Post-quantum authenticated P2P transport    | <- T202, T203, T501       |
| T602 Network anonymity protocol                  | <- T004, T601             |
| T603 Full node, pruning and recovery             | <- T305, T503, T504, T601 |
| T604 End-to-end wallet/network/system validation | <- T405, T602, T603       |

Layer 7 - Independent assurance

|                                              |                           |
|----------------------------------------------|---------------------------|
| T701 Composition review and release decision | <- T104, T305, T504, T604 |
|----------------------------------------------|---------------------------|

This ordering removes the former cycle in which polynomial multiplication depended on an NTT task that itself depended on polynomial multiplication.

## 6. Task specifications

### T001 — Security profile and threat model

**Objective:** Freeze the security games, adversaries, corruption thresholds, privacy boundaries, lifetime, multi-user counts, and minimum composed post-quantum security target.

**Deliverables:**

- cryptographic, consensus, network, side-channel, and operational threat models;
- explicit definition of ledger privacy, recipient-key privacy, and network anonymity;
- per-component and composed security budget;
- quantum collision, preimage, and second-preimage cost models, including time/memory assumptions, multi-target loss, and required digest lengths;
- exclusions such as compromised endpoints, with user-facing claim wording.

**Pass:** independent reviewers agree the model is complete enough to analyze R1–R8. **Reject:** “quantum-safe” or “anonymous” appears without a defined game, adversary, or boundary.

### T002 — Canonical serialization and domain registry

**Objective:** Specify every consensus byte and hash domain before code.

**Deliverables:**

- field-by-field encodings, endian rules, length widths, maxima, rejection behavior, version and chain binding;
- registry for transaction ID, authorization, note, nullifier, Merkle, transcript, block, PoW, KDF, address, and upgrade domains;
- output length and required security property for every hash domain;
- parser limits and cost ordering;
- replay and downgrade rules.

**Pass:** a second team can create the same bytes without reading an implementation. **Reject:** implicit serialization, trailing bytes, unbounded vectors, unknown-version fallback, or one hash reused across purposes.

### T003 — Test-vector and interoperability harness

**Objective:** Build the reproducible owner for official KATs and Quantum cross-language vectors.

**Deliverables:**

- provenance for each external vector;
- canonical JSON or another frozen, integer-safe vector format;
- positive and negative vectors;
- deterministic generator revision and independently checked expected values;
- runner contract for at least two implementations.

**Pass:** clean-room implementations agree byte-for-byte and reject every negative vector. Generated expected values MUST NOT be produced by the same code path being tested without independent confirmation.

## T004 — Benchmark and artifact harness

**Objective:** Make all performance, storage, and network evidence reproducible.

**Deliverables:**

- pinned operating systems, compilers, CPU features, memory, disks, network topology, clock source, and revisions;
- deterministic workload generator with private note creation, proofs, conflicts, malformed traffic, delay, and reorgs;
- raw metrics, logs, hashes, manifests, and report generator;
- accepted-state-transition throughput as the primary metric.

**Pass:** an independent lab reproduces the reported result within a declared tolerance. **Reject:** “latest stable,” unpublished cloud shape, submitted TPS, warm-cache-only measurement, or missing raw artifacts.

## T101 — Field and extension-field arithmetic

**Objective:** Select and implement candidate base and extension fields whose use and security are then approved by T303.

**Deliverables:** canonical field encoding; add/subtract/multiply/invert; extension arithmetic; batch inversion; constant-time policy; reference and optimized implementations; exhaustive small-model and differential tests.

**Pass:** all identities, boundary values, invalid encodings, and cross-language vectors pass; proof-system reviewer confirms the field supports the soundness analysis. **Reject:** a 64-bit base field is presented as 128-bit soundness by itself.

## T102 — Polynomial and NTT layer

**Objective:** Implement polynomial and NTT operations only after the selected commitment or proof profile requires them.

**Deliverables:** ring definition, modulus/root derivation, forward/inverse normalization, naive reference multiplication, optimized multiplication, and aliasing/memory rules.

**Pass:** optimized results equal the independent naive reference across vectors and randomized boundary cases. **Reject:** circular dependency, floating-point roots, undocumented padding, or silent reduction.

## T103 — Samplers and commitment candidate

**Objective:** Implement one precisely cited commitment candidate without inventing a new formula.

**Deliverables:** construction citation and delta from it; exact parameters; matrix/seed generation; canonical encodings; unbiased rejection samplers; commit/open API; note-field binding; deterministic test mode isolated from production entropy.

**Pass:** correctness and vectors pass and the artifact is ready for T104 analysis. **Reject:** byte modulo sampling, reused randomness, invertible-linear “binding” intuition, mismatched randomness length, or carry-dependent value encoding.

## T104 — Commitment security gate

**Objective:** Decide whether the commitment candidate meets R1, R3, and R4.

**Deliverables:** quantum hiding/binding definitions and reduction; concrete parameter estimates with reproducible lattice-estimator inputs; multi-target loss; side-channel review; independent cryptographer report.

**Pass:** composed security remains at least 128 post-quantum bits and all findings are resolved. **Reject:** collision-search testing is used as proof, the construction is only named “BDLOP-like,” or a guessed value can be tested from public data.

## T201 — SHAKE256 and domain hashing

**Objective:** Implement FIPS 202 SHAKE256 and the Quantum QH wrapper exactly.

**Deliverables:** official FIPS 202 KATs; incremental/one-shot behavior; Quantum vectors; tag registry enforcement; output-length binding; per-domain collision/preimage/second-preimage requirement; concrete quantum and multi-target output-length analysis; canonical error behavior.

**Pass:** official and Quantum vectors match in both implementations. **Reject:** SHA3-256 is silently substituted, tags are concatenated ambiguously, or XOF length is unbound or a 32-byte research vector is treated as proof that every collision-dependent consensus digest meets R3.

## T202 — SLH-DSA authorization

**Objective:** Integrate FIPS 205 SLH-DSA-SHAKE-256f.

**Deliverables:** exact FIPS API/context/randomization choices; official KATs; canonical key/signature encodings; authorization digest; key-erasure and side-channel rules; malformed-signature corpus; proof-circuit cost report; decision and concrete-security analysis for the FIPS 205 message-bound signature caveat, including any reviewed transformation.

**Pass:** both implementations match KATs, reject malformed/non-canonical inputs, and the T301/T304 design can afford in-proof verification without weakening R6. **Reject:** protocol IDs say SPHINCS+, a classical signature fallback exists, or signatures authorize a partial transaction.

## T203 — ML-KEM, KDF and authenticated encryption

**Objective:** Define recipient encryption building blocks with ML-KEM-1024.

**Deliverables:** FIPS 203 KATs and errata version; decapsulation-failure behavior; transcript-bound KDF; 256-bit AEAD profile; nonce and replay rules; key separation; receiver-anonymity/key-privacy game for note encryption; multi-user, chosen-key, chosen-ciphertext and cross-output analysis; scanning-tag disclosure budget; test vectors.

**Pass:** chosen-ciphertext, receiver-anonymity and misuse analysis covers the exact note-encryption composition. **Reject:** ML-KEM is treated as authentication, failure creates an oracle, nonce reuse is possible, or an invented Noise pattern name substitutes for a message specification, or the ciphertext identifies its recipient key.

## T204 — Wallet key hierarchy and address profile

**Objective:** Freeze mnemonic import, child-key derivation, address payload, and storage protection.

**Deliverables:** exact BIP-39 behavior; domain-separated post-quantum child derivation; spend/nullifier/encryption key separation; incoming-view, full-wallet-view, transaction-disclosure, and auditor-scoped capability derivation; canonical scope encodings; canonical lower Base32 address vectors; memory-hard wallet-storage KDF; recovery tests.

**Pass:** BIP-39 official vectors and Quantum derivation/address/disclosure vectors match both implementations, and each disclosure capability reveals no more than its stated scope. **Reject:** PBKDF2 parameters are called BIP-39 while using a different salt/normalization, one key crosses protocol roles, or a universal viewing backdoor is required.

## T301 — Integer transaction relation and AIR

**Objective:** Encode authorization, input/output openings, encryption binding, ranges, and exact conservation.

**Deliverables:**

- formal relation before AIR translation;
- public-input and witness schemas;
- constraints for every input commitment opening and membership leaf;
- constraints for every output commitment opening;

- SLH-DSA authorization over the complete digest;
- 16-bit limb range checks and wide carry-constrained sums;
- fee and reward-type separation;
- AIR degree and trace-size report.

**Required negative cases:**

- replace an input value while keeping its commitment;
- spend a one-unit note while claiming a near-maximum input value;
- make both sides equal only modulo the proof field;
- cross a 16-bit carry boundary such as  $65,535 + 1$ ;
- omit or alter an output opening, fee, chain ID, or authorization field.

**Pass:** every negative case is unsatisfiable and reviewers trace each R4 clause to constraints.

**Reject:** balance is only a field equality or an input opening is absent.

### T302 — Membership, nullifier and state AIR

**Objective:** Prove note membership and correct unlinkable nullifier derivation against the exact state hashes.

**Deliverables:** depth and empty-root definitions; leaf/internal tags; membership-index constraints; nullifier relation; local uniqueness; anchor/public-state binding; digest length derived from T001/T201 for Merkle collision and second-preimage requirements; invalid-path and alternate-tree vectors.

**Pass:** all paths and indices are bound, invalid paths fail, and no public field creates a direct note-to-nullifier link. The selected digest length MUST meet R3 after multi-target and lifetime losses. **Reject:** path direction is unconstrained, empty roots differ across implementations, a 32-byte wrapper vector silently becomes the Merkle security parameter, or tree depth exhaustion is ignored.

### T303 — Fiat-Shamir, FRI and zero-knowledge profile

**Objective:** Select the complete transparent proof profile.

**Deliverables:** base/extension fields; transcript schedule; challenge sampling; AIR composition; FRI/DEEP-FRI parameters; query/grinding/batching; zero-knowledge masks; classical/QROM soundness calculations; multi-proof loss; denial-of-service limits.

**Pass:** independent proof-system review shows at least 128 post-quantum bits after composition and no witness leakage under the stated model. **Reject:** the arithmetic “queries  $\times$  bits” is used as soundness, target is  $2^{100}$ , masking is absent, or a generic STARK is assumed zero knowledge.

### T304 — Prover, verifier and aggregation

**Objective:** Implement the T301–T303 profile without accepting ambiguous proofs.

**Deliverables:** independent prover/verifier interfaces; streaming/resource limits; transcript vectors; malformed-proof corpus; deterministic failure codes; individual and, if selected, aggregate wire mode; exact binding of an aggregate to the ordered transactions, public inputs, and pre/post-state; differential/fuzz tests; measured proof generation and verification.

**Pass:** invalid witnesses and mutated proofs fail; both implementations interoperate; aggregate mode omits replaced individual proofs; performance is fed into T305. **Reject:** verifier panics, work is allocated before size checks, both proof modes are redundantly transmitted, or tests are claimed as soundness proof.

### T305 — Cryptographic transaction feasibility gate

**Objective:** Decide whether the complete private transaction is practical before wallet, node, or GHOSTDAG integration consumes implementation effort.

**Required prototype:** exactly two inputs and two outputs with input openings, Merkle membership, nullifier derivation, output openings, encrypted-note binding, complete in-proof SLH-DSA authorisation, 64-bit range constraints, carry-safe conservation, public fee, zero-knowledge masking, and the selected individual and aggregate proof path.

**Deliverables:** two interoperable implementations; pinned desktop and constrained-client hardware; single-wallet p50/p95/p99 proving latency; peak and steady memory; verifier time and memory; proof and transaction bytes; parallel-prover scaling; aggregate latency, size, and amortised verification; raw artifacts and a signed stop/go decision.

**Pass:** every T004 threshold frozen before the run is met without mock signatures, omitted relations, disabled zero knowledge, retained individual proofs in aggregate mode, or unreported preprocessing. **Reject:** the design misses any frozen client, verifier, wire, or aggregate budget. A reject returns the signature, commitment, relation, or proof profile to research; it MUST NOT be deferred to node optimisation.

#### T401 — Note creation and recipient encryption

**Objective:** Create a note that the recipient can discover and later spend without publishing recipient or amount.

**Deliverables:** note plaintext schema; commitment randomness; encryption witness; diversified recipient data; authenticated binding to transaction and output index; receiver-anonymity/key-privacy analysis linked to T001/T203; sender/recipient test vectors; failure and deletion behavior.

An output-creation API MUST receive all data needed by the final note commitment, including spend-authorization and nullifier-key digests. A view public key alone is insufficient.

**Pass:** intended recipients recover exactly the committed note, non-recipients cannot distinguish which eligible recipient key was targeted beyond the approved disclosure budget, and replay/swapping fails. **Reject:** encryption and commitment can describe different notes, or a ciphertext/scanning tag identifies the recipient.

#### T402 — Transaction codec, builder and authorization hash

**Objective:** Construct and parse the exact public transaction while keeping private witnesses off-ledger.

**Deliverables:** canonical codec; maximum counts/sizes; dedicated sighash; builder state machine; duplicate handling; expiry/anchor/proof-mode binding; negative parser corpus.

**Pass:** round trips are byte-stable, alternate encodings fail, and every mutable semantic field changes the authorization digest. **Reject:** parser fallbacks, unbounded allocation, signature malleability, or unknown fields silently ignored.

#### T403 — Commitment/nullifier state and atomic application

**Objective:** Implement the single owner for note-tree, nullifier-set, and state-root transitions.

**Deliverables:** atomic apply/revert; deterministic roots; duplicate nullifier checks; snapshot/recovery format; reorg journal; concurrency model; invariants and crash tests.

Transaction verification MUST take an explicit immutable state/anchor context. A stateless `Transaction::verify()` is insufficient for global nullifier and anchor checks.

**Pass:** crashes and reorgs recover to the same root, two spends cannot both commit, and no non-owner bypasses state validation. **Reject:** check-then-write races, partial state commits, or hidden empty-state fallback.

#### T404 — Wallet scanning and spending

**Objective:** Build deterministic private wallet behavior over T401–T403.



**Deliverables:** scan/decrypt pipeline; note and nullifier tracking; spend selection; proof/sign/build flow; reorg rollback; scoped disclosure export and receipt verification; disclosure limitations; secure deletion; user-facing finality and fee semantics.

**Pass:** restore-from-seed and rescan reproduce the same wallet state through reorgs; spent notes cannot be selected; no network identity is derived from wallet keys. **Reject:** missing data is zero-filled, failed decryption is treated as an empty note, or sensitive logs are emitted.

#### T405 — Selective disclosure and bounded payment policies

**Objective:** Add useful settlement controls without introducing a general virtual machine, a universal viewing key, or an unbounded proof relation.

**Deliverables:** incoming-view, full-wallet-view, transaction-specific, and auditor-scoped disclosure flows; canonical scope and receipt encodings; least-privilege and unlinkability tests; documented non-revocation of copied history; a finite registry of accepted payment policies; exact semantics and costs for each enabled timelock, hashlock, threshold/multisignature, escrow, recurring-authorisation, atomic-swap, or conditional-release policy; updated AIR/proof profile, vectors, and resource report for every enabled policy.

Private asset issuance and cross-chain adapters are separate post-core profiles. Any proposed adapter MUST publish its external consensus, custody, MPC/multisignature, oracle, light-client, finality, privacy, and upgrade trust assumptions; it does not inherit Quantum’s base-layer assurance.

**Pass:** disclosure never exceeds its canonical scope, policy execution is deterministic and bounded across reorgs/timeouts, and every enabled policy preserves transaction privacy, authorisation, supply, and T305 performance budgets. **Reject:** a global backdoor key exists, copied disclosure is called revocable, unknown policy bytecode falls back to execution, a policy can mint or bypass authorisation, or an adapter is described with stronger properties than its own evidence supports.

#### T501 — Versioned GHOSTDAG consensus profile

**Objective:** Turn the cited GHOSTDAG research into one deterministic, versioned protocol profile.

**Deliverables:** parent validation; anticone and blue/red rules; selected parent; merge ordering; blue work; finality/pruning assumptions; block limits; reference model; exhaustive small-DAG and adversarial vectors.

**Pass:** independent implementations produce identical classification, ordering, score/work, and finality across all vectors. **Reject:** “highest blue score wins,” a simple topological sort, or upstream behavior without a pinned version is treated as the specification.

#### T502 — Header, proof of work and deterministic DAA

**Objective:** Specify and implement canonical headers, PoW, and expected difficulty.

**Deliverables:** exact header bytes and size; block/PoW domains; integer target conversion; DAA equations, window, clamps and overflow rules; timestamp rules; genesis-independent vectors; time-warp simulations.

**Required negative cases:**

- miner supplies an easy target;
- nodes have different local clocks;
- timestamps are reordered or clamped;
- maximum/minimum targets and arithmetic boundaries;
- semantically equal headers use alternate bytes.

**Pass:** validators recompute and enforce the same expected target without floating point or local-time dependence. **Reject:** the header target is only used to check its own PoW.

### T503 — Canonical DAG-to-state ordering

**Objective:** Prove and implement one deterministic state transition order for parallel blocks.

**Deliverables:** ordered merge-set algorithm; transaction ordering; duplicate and concurrent-nullifier rule; pre/post-state commitments; reorg semantics; model checker or equivalent state exploration; adversarial DAG vectors.

**Pass:** all implementations converge on accepted transactions and state root under permutations, delays, conflicts, and reorgs. **Reject:** parallel blocks mutate shared state concurrently or conflict resolution depends on arrival order.

### T504 — Rewards, supply cap and genesis

**Objective:** Define value creation and the immutable network start.

**Deliverables:** distinct reward transaction; DAA-score or other exact DAG issuance index; blue/red/state eligibility; integer schedule and rounding; cumulative cap proof; exact fee-transfer, subsidy-issuance, burn and note-supply equations; reward maturity and reorg reversal; private reward-output handling; canonical genesis bytes and vectors.

**Pass:** exhaustive/bounded-model checks and analytical proof show issuance never exceeds 21,000,000 QTM across reorganizations, fees are never counted as new issuance, and all nodes derive identical rewards. **Reject:** linear height is used without a DAG definition, a fee can be claimed twice or counted as subsidy, no coinbase/reward transaction exists, timestamp units differ, or genesis hashes with another function.

### T601 — Post-quantum authenticated P2P transport

**Objective:** Define link confidentiality/authentication without classical fallback or wallet identity leakage.

**Deliverables:** exact handshake message sequence; ML-KEM/SLH-DSA transcript binding; KDF/AEAD keys; replay/downgrade/key-compromise analysis; peer identity lifecycle; vectors and state-machine fuzzing.

**Pass:** independent cryptographic review approves the exact composition and implementations interoperate. **Reject:** “post-quantum Noise” is used without an actual reviewed pattern, ML-KEM is mistaken for authentication, or failed PQ negotiation falls back to X25519.

### T602 — Network anonymity protocol

**Objective:** Meet R2 against the T001 observer rather than merely encrypt links.

**Deliverables:** routing and cover-traffic protocol; peer/route selection; batch/delay distributions; active tagging, eclipse, Sybil and intersection analysis; simulator; reproducible attack experiments; utility/privacy tradeoff report.

**Metrics:** source-classification advantage, anonymity-set distribution, entropy, precision/recall for origin inference, latency, bandwidth amplification, failure behavior, and confidence intervals.

**Pass:** thresholds defined in T001 are met under all required scenarios and an independent network-privacy review passes. **Reject:** only ciphertext byte uniformity, payload correlation, or Dandelion++ deployment is offered as proof of full anonymity.

### T603 — Full node, pruning and recovery

**Objective:** Integrate consensus, proof verification, atomic state, transport, storage, and resource admission.

**Deliverables:** validation pipeline with cheap checks first; bounded queues; peer resource accounting; state persistence; pruning/snapshot/archive model; trustless recovery; reorg handling; observability without sensitive data; fault-injection results.

**Pass:** state roots remain identical under crash, restart, overload, reorder, and recovery; no malformed input causes unbounded work. **Reject:** hidden fallback state, partial validation, archive size presented as pruned-node size, or snapshot trust is undocumented.

## T604 — End-to-end wallet/network/system validation

**Objective:** Demonstrate R1–R8 together, not in isolated microbenchmarks.

**Required run:**

- geographically distributed topology;
- private note creation, scanning, spending, proofs, propagation and state application enabled;
- selective-disclosure flows and every payment policy enabled in the tested release profile;
- at least 1,000 accepted layer-1 tx/s for 24 hours;
- conflict, delay, peer loss, malformed traffic, reorg, restart and recovery phases;
- raw proof-size, bandwidth, storage, CPU, memory, latency, rejection and anonymity metrics.

**Pass:** all functional invariants remain true, the throughput target is sustained, the anonymity thresholds pass, and another team reproduces the report. **Reject:** benchmark uses mock proofs, prevalidated transactions, disabled privacy, submitted TPS, a single process, or unpublished hardware.

## T701 — Composition review and release decision

**Objective:** Decide whether the combined system—not only its parts—meets the three non-negotiable requirements.

**Deliverables:** requirements-to-evidence manifest; independent cryptographic, proof, consensus, network, implementation and performance reports; finding register; per-capability claim register; residual-risk statement; legal review; signed go/no-go record.

**Pass for testnet:** all blocking design gates pass, two implementations interoperate, and no unresolved critical/high issue permits inflation, unauthorized spend, privacy break, consensus divergence, or target bypass.

**Pass for production:** testnet evidence remains valid over the approved stability period, external audits are closed, governance/upgrades are post-quantum and replay-safe, operational recovery is proven, and public claims match evidence.

**Reject:** an author or AI self-certifies the system, a component review is presented as composition review, or any hard requirement is deferred until after launch.

## 7. Mandatory adversarial scenarios

Every relevant implementation must preserve a permanent regression test for:

| ID   | Scenario                                                           | Required outcome                       |
|------|--------------------------------------------------------------------|----------------------------------------|
| A001 | Small real input opened with a larger private input value          | Proof unsatisfiable                    |
| A002 | Input/output balance equal only modulo the proof field             | Proof unsatisfiable                    |
| A003 | 65,535 + 1 crosses an unconstrained limb carry                     | Correct integer result only            |
| A004 | Output commitment and encrypted output describe different notes    | Rejected                               |
| A005 | Authorization omits fee, output, chain, anchor, or proof mode      | Rejected                               |
| A006 | Same nullifier in one transaction, block, parallel block, or reorg | At most canonical first spend accepted |
| A007 | Header declares an easier target than the DAA computes             | Block rejected                         |
| A008 | Honest nodes use different local clocks on the same DAG            | Identical consensus result             |
| A009 | Equivalent DAG delivered in different arrival orders               | Identical ordering and state root      |

| ID   | Scenario                                                                                            | Required outcome                                                           |
|------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| A010 | Oversized vectors/proofs or unknown version                                                         | Rejected before expensive work                                             |
| A011 | Failed PQ negotiation requests<br>classical fallback                                                | Connection rejected                                                        |
| A012 | Origin observer combines timing, volume and route metadata                                          | Must remain below T001 attack threshold                                    |
| A013 | Snapshot interrupted or maliciously altered                                                         | Recovery rejects or reaches canonical root                                 |
| A014 | Reward rounding/reorg approaches the supply cap or replays fees                                     | Cap never exceeded; fee portion never counted as issuance or claimed twice |
| A015 | Observer compares note ciphertexts or scanning tags against candidate recipient keys                | Recipient inference remains below the T001 threshold                       |
| A016 | Disclosure capability is replayed outside its account, counterparty, transaction, or time scope     | No additional note or wallet history is revealed                           |
| A017 | Unknown payment-policy identifier, unbounded policy input, timeout/reorg edge, or adapter downgrade | Fail closed; deterministic bounded state; no inherited bridge claim        |

These are minimum cases, not a complete security test suite.

## 8. Performance and capacity report

Every benchmark report **MUST** contain:

1. repository revisions and build hashes;
2. exact dependency/toolchain versions;
3. machine and network inventory;
4. workload seed and transaction-shape distribution;
5. accepted, rejected, duplicate, and submitted counts separately;
6. p50/p95/p99 proof, propagation, ordering, application and finality latency;
7. average/p95/max transaction and proof bytes;
8. per-node inbound/outbound bandwidth and gossip amplification;
9. CPU, memory, disk I/O, state and archive growth;
10. anonymity metrics and added bandwidth/latency;
11. all failures, restarts, divergences and discarded samples;
12. commands and immutable artifact locations.

T305 reports **MUST** additionally separate single-wallet latency from parallel prover throughput and aggregate amortisation. A server aggregate rate **MUST NOT** be reported as client-side transaction latency.

The 1,000 tx/s acceptance target is fixed. Reference hardware and proof-latency thresholds are intentionally not fabricated in advance; T004 must freeze them before a result can be interpreted. A 60-second benchmark limit **MUST NOT** be quietly substituted for a different specification limit.

## 9. Security review checklist

### Cryptographic correctness

- ☐ FIPS 202, 203, and 205 algorithm names and parameter sets are exact.

- ☐ Official KAT provenance is recorded.
- ☐ Every hash domain has a security property and output length justified against quantum and multi-target attacks.
- ☐ Commitment construction and parameters pass T104.
- ☐ Every random sampler is unbiased and domain-separated.
- ☐ No secret-dependent branch, memory lookup, log, or error oracle remains.
- ☐ Multi-user and protocol-lifetime composition remains 128 PQ bits.

### **Transaction soundness**

- ☐ Each input value is bound to an opened member commitment.
- ☐ Each output value is bound to its public output commitment.
- ☐ Authorization key is bound to the spent note.
- ☐ Authorization digest covers every semantic field.
- ☐ All ranges and sums use constrained integer limbs and carries.
- ☐ Reward minting is type-separated from ordinary conservation.
- ☐ Global anchor/nullifier checks use explicit state context.

### **Privacy and anonymity**

- ☐ No transparent transfer mode exists.
- ☐ Public ledger fields meet the R1 disclosure budget.
- ☐ Note encryption and scanning metadata meet the recipient-key-privacy game.
- ☐ Note ciphertext and commitment cannot be swapped or replayed.
- ☐ Wallet, transport, and transaction identities are separated.
- ☐ T602 covers global passive and required active attacks.
- ☐ Incoming, full-wallet, transaction-specific, and auditor-scoped disclosure are distinct and least-privilege.
- ☐ Copied disclosure is not described as revocable or as proof of legal compliance.

### **Product boundary and performance**

- ☐ T305 passes before full-node or GHOSTDAG integration starts.
- ☐ The 2-input/2-output prototype includes every required relation and in-proof SLH-DSA verification.
- ☐ Wallet latency, parallel proving, verifier cost, wire size, and aggregate amortisation are reported separately.
- ☐ Every enabled payment policy has bounded deterministic semantics and an updated proof/resource report.
- ☐ No general-purpose VM, private asset, bridge, or adapter inherits a base security, privacy, or compliance claim.

### **Consensus and supply**

- ☐ GHOSTDAG profile is executable and versioned.
- ☐ DAA is deterministic integer arithmetic.
- ☐ Validators recompute expected target.
- ☐ DAG ordering yields one atomic state history.
- ☐ Reward eligibility and index are DAG-native.
- ☐ Fee transfer, subsidy issuance, burns and reward maturity are exact.
- ☐ Cap proof includes fee replay, rounding and reorganization.
- ☐ Genesis bytes and timestamp units are canonical.

### **Parsing and operations**

- ☐ All lengths and counts are bounded.

- ☐ Unknown versions and non-canonical encodings fail closed.
- ☐ Cheap validation precedes expensive cryptography.
- ☐ Queues, memory, CPU and disk work are bounded per peer.
- ☐ Snapshots are authenticated and trustless recovery is tested.
- ☐ Logs and crash artifacts contain no wallet or network-origin secrets.

Unchecked boxes are normal at this research stage. They MUST NOT be prechecked in reports.

## 10. Evidence manifest and report format

Each task produces a record with:

```
task_id: T301
status: NOT_STARTED # NOT_STARTED / IN_PROGRESS / BLOCKED / PASS / FAIL
owner: null
reviewer: null
requirements: [R1, R3, R4]
source_revision: null
toolchain_manifest: null
evidence:
  analytical_argument: null
  formal_artifact: null
  test_report: null
  vector_set: null
  benchmark_artifacts: null
  external_review: null
open_findings: []
decision_date: null
decision_signature: null
```

Null fields mean **not supplied**, never “not needed” or “passed.” PASS is allowed only when every mandatory artifact for that task exists and the named reviewer signs the exact revision.

### Current aggregate report

| Area                        | Status      | Evidence                            |
|-----------------------------|-------------|-------------------------------------|
| Requirements draft          | IN_PROGRESS | Specification 0.3.0-research        |
| Commitment                  | NOT_STARTED | No selected construction            |
| Proof profile               | NOT_STARTED | No frozen parameters or AIR         |
| Private transaction         | NOT_STARTED | Relation drafted; no implementation |
| DAG/state integration       | NOT_STARTED | Requirements only                   |
| Network anonymity           | NOT_STARTED | Requirements only                   |
| 1,000 accepted tx/s         | NOT_STARTED | No benchmark                        |
| Independent implementations | NOT_STARTED | None                                |
| External audit              | NOT_STARTED | None                                |

## 11. Proposed implementation workspace

The following layout is a proposal for a future implementation repository; it does not exist here and is not a command contract:

```
quantum-protocol/
  spec/                canonical encodings and generated vectors
  reference/           simple independent reference implementation
  node/               production-oriented validator implementation
  wallet/             wallet, scanner and proof client
```

```

formal/                models and formal artifacts
tests/
  kats/
  interoperability/
  adversarial/
  fuzz/
  performance/
evidence/              immutable manifests and signed reports

```

Build, test, fuzz, benchmark, and vector commands MUST be defined by that repository and pinned in T003/T004. This documentation repository intentionally does not publish fictitious commands that appear runnable.

## 12. Release labels

Only these labels may be used:

| Label                      | Minimum meaning                                            |
|----------------------------|------------------------------------------------------------|
| Research design            | Requirements and candidates; blocking gates may be open    |
| Prototype                  | Non-production code; no security or performance claim      |
| Reviewed testnet candidate | T701 testnet pass on an exact revision                     |
| Public testnet             | Candidate deployed with explicit residual risks            |
| Production candidate       | T701 production evidence complete; no live claim yet       |
| Production                 | Exact audited revision deployed and continuously monitored |

At the current revision the project is a **research design**.

## 13. Revision record

### 0.3.0-research — 2026-07-21

- expanded the acyclic plan to 31 tasks with T305 as the mandatory complete transaction feasibility gate before node/consensus integration;
- separated single-wallet proof latency, parallel prover throughput, aggregate amortisation, and verifier/wire budgets;
- added T405 for least-privilege disclosure and a finite, proof-bounded payment policy registry;
- made private assets and interoperability adapters separate post-core profiles with explicit external trust and claim boundaries;
- added adversarial cases for disclosure-scope escalation and unknown or unbounded payment-policy behavior.

### 0.2.0-research — 2026-07-09

- changed the guide from autonomous AI implementation instructions to a human-owned research and verification plan;
- defined all 29 task IDs and removed missing-task references;
- made the dependency graph acyclic;
- added requirements traceability and honest NOT\_STARTED reporting;
- added direct regression cases for input-value inflation, field wrap, self-declared PoW targets, local-time DAA divergence, DAG conflicts, and anonymity correlation;
- distinguished proof, formal verification, testing, statistics, benchmarks, review, and audit;
- aligned performance acceptance with 1,000 end-to-end accepted tx/s;

- required external specialist and composition review before any release claim.
- added explicit gates for quantum digest lengths and encrypted-note recipient-key privacy;
- defined fee transfer, claimed subsidy, burns, maturity, and reorg accounting as mandatory supply evidence.